

3. When an object is removed from a container like a list, tuple, or dictionary, for example with:

```
myList.pop()
```

Calling the `remove()` method of a list would also decrement the reference count.

4. When you use the `del` statement to explicitly delete a variable that refers to an object. This eliminates the variable and reduces the reference count of the object:

```
del oSquare3 # delete the variable
```

5. If the reference count of the object's container (in this case, `myList`) goes to zero:

```
del myList # where myList has an element that refers to an object
```

If you have a variable that refers to an object, and you want to keep the variable but lose the reference to the object, you can execute a statement like the following:

```
oSquare1 = None
```

That keeps the variable name, but lowers the reference count of the object.

Death Notice

When the reference count of an object goes to zero, Python knows that the object can be safely deleted. Right before destroying an object, Python calls a magic method of that object named `__del__()` to inform the object of its impending demise.

In any class, you can write your own version of the `__del__()` method. In your version, you can include any code you want your object to execute (ha! interesting word choice here) before the object disappears forever. For